Reproducibility

Today's agenda:

- Making code reproducible
- R Markdown
- Work on .Rmd files

Reproducibility

When using code, this means:

- Analyses could be re-run and would give the same results
- Code and data are available and usable

Reproducibility is a spectrum:

- Poorly structured code and data are less reproducible

Elements of reproducibility: the 4 D’s

Design (structure and style)

Documentation

Dependencies

Data

Deposit

Design

Well thought-out folder structure

E.g., folders for:

- Functions
- Scripts
- Raw Data
- Clean Data
- Figures

Design

Well thought out code

- If you do something multiple times, make it a function
- Make code modular
- Break up scripts into different files (e.g., data cleaning, data analyses, etc.)

Design

Use a consistent code style

- We discussed this previously when talking about tidyverse
- We also discussed R function to help style your code

Documentation

Use comments!

- Document what different sections do
- Document any decisions or assumptions
- Explain complicated bits of code

Documentation

Include readme files

- One at the project's root directory
  - Explain the purpose
  - Is there anything special needed to know to run the code?
  - Can be relatively simple or very complex and detailed
  - Can contain a link to any resulting publications
- Other readme files where needed
  - Add readme files in any folder where information is helpful
  - Example: noting data sources, file naming structure, etc.
  - Readme files that accompany datasets are often very helpful

Documentation

Document functions

- You can document any functions you write using comments
- R also has a special way of documenting functions: Roxygen comments
- Example (edited to fit on slide):

```
#' @name dr_rulsif
#' @title Density-ratio SDM estimation with RuLSIF
#' @description dr_rulsif is an internal function for density-ratio estimation with RuLSIF
#' @param presence_data dataframe of covariates
#' @param background_data dataframe of covariates
#' @importFrom densratio RuLSIF
#' @importFrom Rdpack reprompt
#' @keywords internal
#' @references
#' \insertAllCited{}
```

Documentation

Document functions

Include things like:

- A description of what they do
- What the parameters are and the expected data types
- What the function returns
- What functions it relies on (if any)
- An example of how to use it

Dependencies

- Your code relies upon the R program as well as various packages
- R and R packages change over time
- Code written at one time may not work in the future as things change
- It's useful to note which versions of things you use

Dependencies

The renv package is designed to manage dependencies for you

- Keeps track of which packages you use and which versions
- Stores the packages inside our directory
- Set up with `renv::init()`
- Update with `renv::shapshot()`
- See https://rstudio.github.io/renv/ for details

Dependencies

If you REALLY want to be reproducible, you can use containers

- Package all the code you need to run analyses
- Even include a lightweight operating system
- This means you can run the same exact code on any OS
- Prevents OS-related issues
- Also useful if you want to work on a cluster

Data

- Keep raw data untouched, seperate from clean data
- Use scripts for cleaning so that it can be replicated
  - Comments in cleaning scripts document your decisions and why you made them
- Use relative paths to load data
- Data should be accessible to others

Data

- Data should be permanently archived somewhere on the internet
- Code ideally downloads the data from the internet
- Small amounts of data can be put on Github
- Large datasets need specialized storage locations, e.g.:
  - OSF (https://osf.io/)
  - Dryad (https://datadryad.org/)
  - FigShare (https://figshare.com/)

Deposit

Like data, code should be deposited in a permanently archived location

- Github is a good step, but isn’t permanent
- Zenodo (https://zenodo.org/) can permanently archive Github code easily
- Zenodo assigns a DOI, so code can be cited

Elements of reproducibility: the 4 D’s

Design (structure and style)

Documentation

Dependencies

Data

Deposit

Other ways to help with reproducibility

- For small bits of code, reprex are useful
- You can also make use of Rmarkdown and Quarto (.rmd and .qmd) files

R markdown

Details at: https://rmarkdown.rstudio.com/lesson-1.html

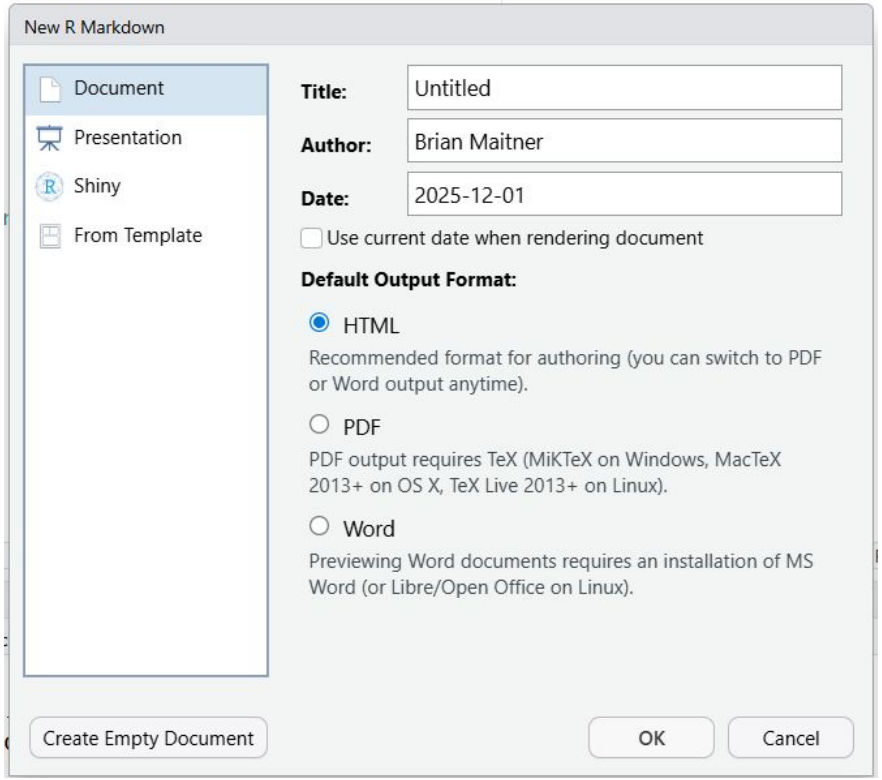
Cheatsheet at: https://rstudio.github.io/cheatsheets/html/rmarkdown.html

- Runs code and produces documents
- Can be fully reproducible
- Can call other programs (e.g., python)

R markdown

Let’s play with R markdown a bit

- First, create a new Rmd file
- In RStudio:
  - File > New File > R Markdown

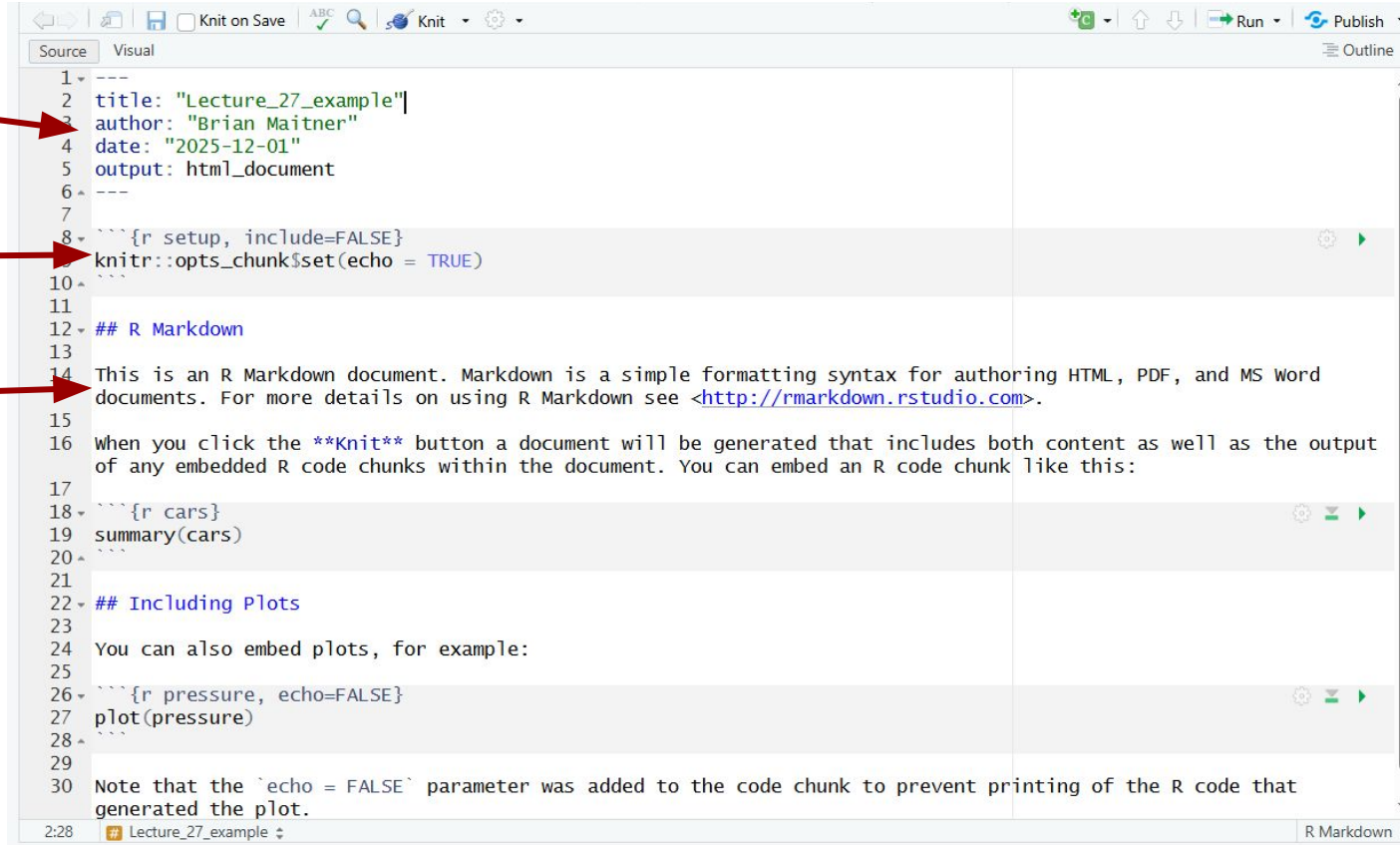


R markdown: 3 types of content

Yaml header

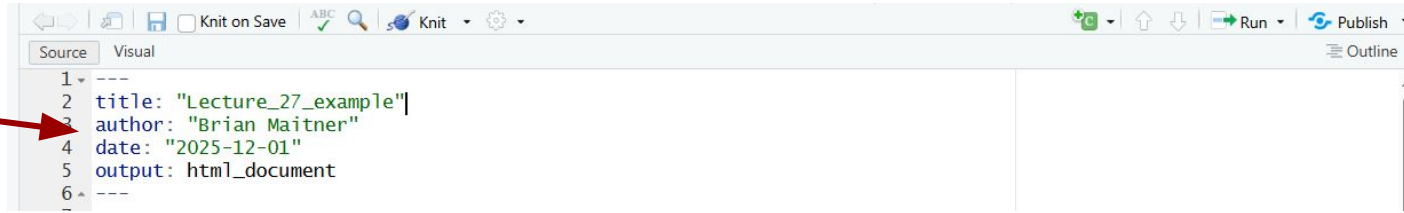
R code block

Markdown text



R markdown: Yaml header

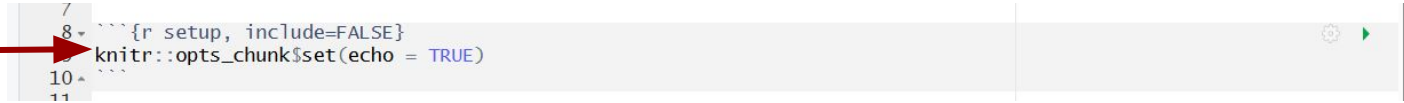
Yaml header



- Contains metadata on the file
- Also contains information on how to render the file
  - Output tells it to render as an html_document
  - Could also render as PDF, .docx, etc.
- There are other advanced option like:
  - Adding a table of contents
  - Incorporating a citations
  - Changing the page margins
- https://rstudio.github.io/cheatsheets/html/rmarkdown.html#chunk-options

R markdown: code blocks

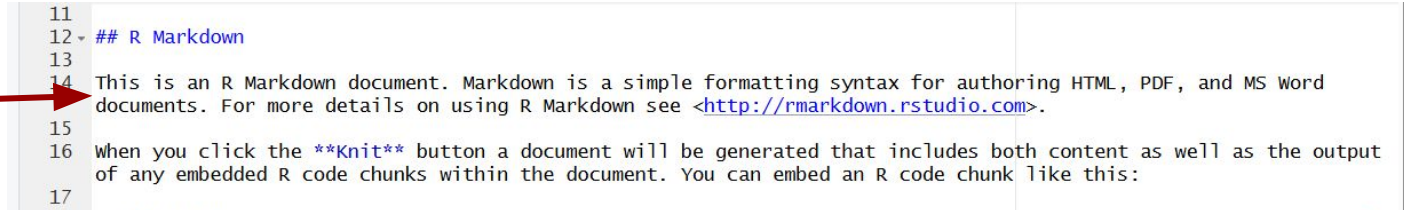
R code block



- Can have as many as you like
- Begin with ```{r}
- End with ```
- Optional arguments go inside {r}
  - Specify whether messages and warnings should be shown
  - Tell R not to run the chunk
  - Specify figure sizes and captions
- https://rstudio.github.io/cheatsheets/html/rmarkdown.html#chunk-options

R markdown: Markdown text chunks

Markdown text



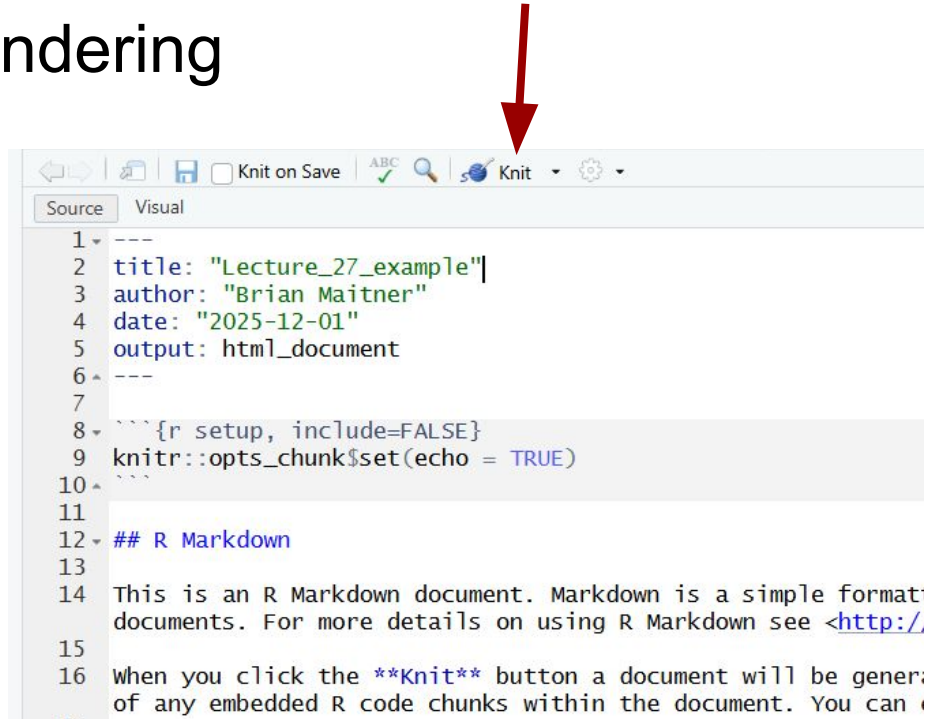
```
11
12 ## R Markdown
13
14 This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word
   documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.
15
16 When you click the **Knit** button a document will be generated that includes both content as well as the output
   of any embedded R code chunks within the document. You can embed an R code chunk like this:
17
```

- Can have as many as you like
- Syntaxy to specify how to render text, e.g.,
  - #, ##, ## denote section headers
  - **text** italicizes
  - * Adds a bullet point
- https://rstudio.github.io/cheatsheets/html/rmarkdown.html#write-with-markdown

R markdown: Rendering

rmarkdown::render()

Click “knit” button



Markdown practice

For the remainder of class I want you to try making your own Rmarkdown files

- Create a blank R markdown file (or use the one we created earlier)
- Populate it with content copied from a previous R script
  - Comments become text blocks
  - Code is placed in code chunks
- Once you've got something that looks good, try rendering it!

Before next class:

- Read:
  - https://doi.org/10.1126/science.aab2374
  - https://doi.org/10.1002/ece3.70030
- May be a quiz